Univariate Linear Regression Code

Model Representation

Notation	Ý nghĩa	Python (nếu có)
a	scalar (một số, **không** in đậm)	
a (hoa)	vector (một vector, **in đậm**)	
x	Training example feature values (ví dụ trong lab: Size – 1000 sqft)	`x_train`
y	Training example targets (ví dụ trong lab: Price – 1000s of dollars)	`y_train`
x, y với i nhỏ viết trên đầu	training example thứ i	`x_i`, `y_i`
m	số lượng training examples	`m`
w	parameter: weights	`w`
b	parameter: bias	`b`

- Import thư viện, numpy để sử dụng tính toán vector, matplotlib.pyplot để vẽ các dữ liệu và đồ thị

```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('./deeplearning.mplstyle')
```

- Phần lab này minh hoạ ví dụ về dự đoán giá nhà, và mình chỉ có 2 training set:

```
# x_train is the input variable (size in 1000 square feet)
# y_train is the target (price in 1000s of dollars)
x_train = np.array([1.0, 2.0])
```

```
y_train = np.array([300.0, 500.0])
print(f"x_train = {x_train}")
print(f"y_train = {y_train}")
```

- Để mình tính được số lượng training example mình sẽ dùng . shape của thư viện numpy, nhưng do x_train là một scalar, nên nó sẽ return về một tuple dạng (m,) khi gọi, mình sẽ lấy index đầu tiên ra [0], đó chính là số training example.

```
# m is the number of training examples
print(f"x_train.shape: {x_train.shape}")
m = x_train.shape[0]
print(f"Number of training examples is: {m}")
```

- Ngoài ra mình cũng có thể sử dụng len() nếu muốn

```
# m is the number of training examples
m = len(x_train)
print(f"Number of training examples is: {m}")
```

- Để ký hiệu số training example mình sẽ dùng x_i và y_i, với i là số training example

```
i = 1 # Change this to 1 to see (x^1, y^1)

x_i = x_train[i]
y_i = y_train[i]
print(f"(x^({i}), y^({i})) = ({x_i}, {y_i})")
```

- Phần code này là để vẽ hình minh hoạ
  - .scatter là hàm vẽ điểm, nhận vào 2 array của x và y, các tham số marker, color để điều chỉnh biểu tượng điểm (chữ x), và màu sắc (r là red)
  - .title đặt tiêu đề
  - .ylabel và .xlabel đặt tiêu đề cho trục y và x tương ứng
  - .show() hiện biểu đồ

```
# Plot the data points
plt.scatter(x_train, y_train, marker='x', c='r')
# Set the title
plt.title("Housing Prices")
# Set the y-axis label
plt.ylabel('Price (in 1000s of dollars)')
# Set the x-axis label
plt.xlabel('Size (1000 sqft)')
plt.show()
```

Model Function:

- Trong bài này mình sẽ tự động chọn 2 tham số w và b, nếu mình muốn đổi param thì sẽ đổi ở đây

```
w = 200
b = 100
print(f"w: {w}")
print(f"b: {b}")
```

Với Model:

$$f_{w,b}(x^{(i)}) = wx^{(i)} + b \tag{1}$$

- Mình sẽ viết một hàm để tính output của model:
  - .zeros tạo ra một numpy array chỉ toàn số 0
  - lặp qua từng training example và tính ra prediction với trọng số và bía tương ứng

```
def compute_model_output(x, w, b):
    """
    Computes the prediction of a linear model
    Args:
      x (ndarray (m,)): Data, m examples
      w,b (scalar)    : model parameters
```

```
    Returns
      y (ndarray (m,)): target values
    """
    m = x.shape[0]
    f_wb = np.zeros(m)
    for i in range(m):
        f_wb[i] = w * x[i] + b

    return f_wb
```

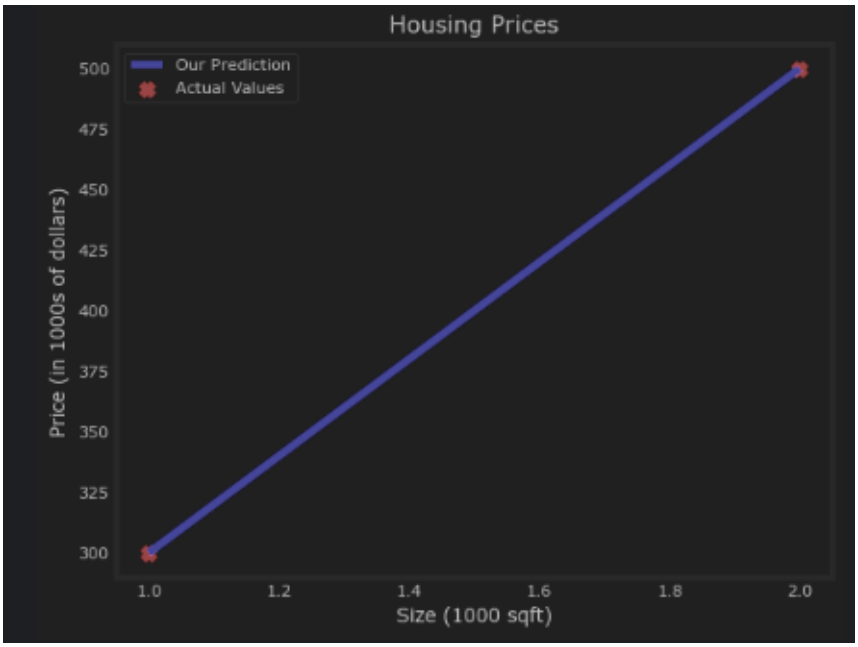
- Đoạn này dưới sẽ sử dụng hàm vừa viết và vẽ nó ra
  - .plot là vẽ thành đường nét (còn scatter là điểm)
  - .legend() là bổ sung thêm phần chú thích

```
tmp_f_wb = compute_model_output(x_train, w, b,)

# Plot our model prediction
plt.plot(x_train, tmp_f_wb, c='b',label='Our Prediction')

# Plot the data points
plt.scatter(x_train, y_train, marker='x', c='r',label='Actual
Values')

# Set the title
plt.title("Housing Prices")
# Set the y-axis label
plt.ylabel('Price (in 1000s of dollars)')
# Set the x-axis label
plt.xlabel('Size (1000 sqft)')
plt.legend()
plt.show()
```



- Khi mình đã xác định được là mô hình chạy đúng với training example, có thể chạy thử một input bên ngoài: (với x_i = 1.2)

```
w = 200
b = 100
x_i = 1.2
cost_1200sqft = w * x_i + b

print(f"${cost_1200sqft:.0f} thousand dollars")
```

→ 340 nghìn đô

Cost Function

Phần lab này sẽ nói về cost function của linear regression 1 biến

- Import thư viện
- Bài này cũng sử dụng một số thư viện được viết sẵn cho khoá học.

```
import numpy as np
%matplotlib widget
import matplotlib.pyplot as plt
from lab_utils_uni import plt_intuition, plt_stationary, plt_
update_onclick, soup_bowl
plt.style.use('./deeplearning.mplstyle')
```

- Vẫn cùng với dữ liệu từ lab trước(về dự đoán giá nhà), ta có 2 numpy array

```
x_train = np.array([1.0, 2.0])          #(size in 1000 squar
e feet)
y_train = np.array([300.0, 500.0])          #(price in 1000s
of dollars)
```

- Ta nhắc lại hàm cost function:

$$J(w,b) = \frac{1}{2m} \sum_{i=0}^{m-1} (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

với:

$$f_{w,b}(x^{(i)}) = wx^{(i)} + b$$

Ta có hàm tính cost:

```
def compute_cost(x, y, w, b):
    """
    Computes the cost function for linear regression.

    Args:
      x (ndarray (m,)): Data, m examples
      y (ndarray (m,)): target values
      w,b (scalar)    : model parameters

    Returns
```

```
        total_cost (float): The cost of using w,b as the para
meters for linear regression
               to fit the data points in x and y
    """
    # number of training examples
    m = x.shape[0]

    cost_sum = 0
    for i in range(m):
        f_wb = w * x[i] + b
        cost = (f_wb - y[i]) ** 2
        cost_sum = cost_sum + cost
    total_cost = (1 / (2 * m)) * cost_sum

    return total_cost
```

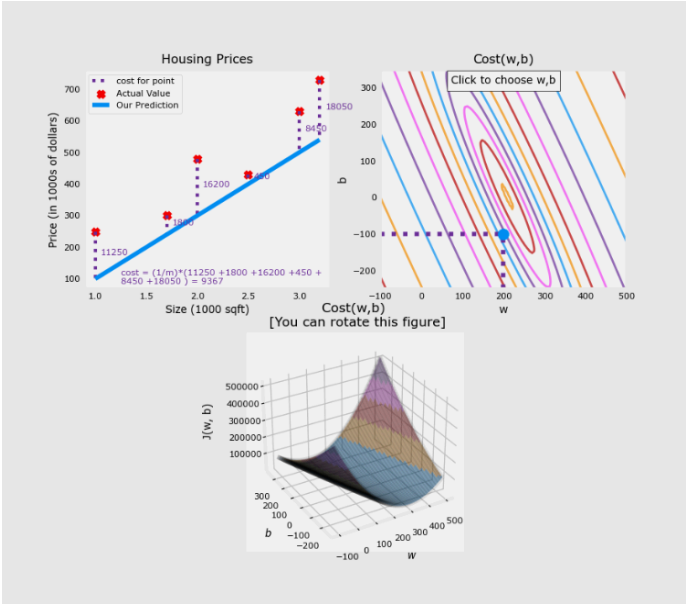
Note:

- m là biến lưu số training examples,
- Ta lặp lại qua từng example một, tính Model sau đó bình phương lên, nhân với (1/2m) và cộng tổng vào biến total_cost.

Giờ mình sẽ thử với một data set lớn hơn, vẫn là 2 numpy array:

```
x_train = np.array([1.0, 1.7, 2.0, 2.5, 3.0, 3.2])
y_train = np.array([250, 300, 480,  430,   630, 730,])
```

Sau đó mình sử dụng một số hàm viết riêng của khoá học để minh hoạ ra Cost trong không gian 3 chiều, cùng với các Contours:



- Ở trong lab phần này mình có thể bấm chọn trên biểu đồ các điểm nơi mà cost nhỏ nhất một cách thủ công. (Địa điểm nơi cost nhỏ nhất là: w=209, b=2.4)

Gradient Descent

Ở phần lab bên trên mình đã lựa chọn w và b một cách thủ công để tìm ra cost nhỏ nhất, giờ mình sẽ sử dụng code Gradient Descent để tự động hoá quá trình này.

- Các thư viện

```
import math, copy
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('./deeplearning.mplstyle')
from lab_utils_uni import plt_house_x, plt_contour_wgrad, plt
_divergence, plt_gradients
```

- Ta có training data của 2 lab trên:

```
# Load our data set
x_train = np.array([1.0, 2.0])   #features
y_train = np.array([300.0, 500.0])   #target value
```

- Cũng như là hàm tính cost đã được viết ở trên:

```
#Function to calculate the cost
def compute_cost(x, y, w, b):

    m = x.shape[0]
    cost = 0

    for i in range(m):
        f_wb = w * x[i] + b
        cost = cost + (f_wb - y[i])**2
    total_cost = 1 / (2 * m) * cost

    return total_cost
```

Nhắc lại một số kiến thức về Gradient Descent, ta có cost function:

$$J(w,b) = \frac{1}{2m} \sum_{i=0}^{m-1} (f_{w,b}(x^{(i)}) - y^{(i)})^2 \tag{2}$$

- Gradient Descent là việc lặp đi lặp lại vòng lặp này cho tới khi cost đạt cực tiểu:

$$\begin{aligned} &\text{repeat until convergence: } \{ \\ &\quad w = w - \alpha \frac{\partial J(w,b)}{\partial w} \\ &\quad b = b - \alpha \frac{\partial J(w,b)}{\partial b} \\ &\quad \} \end{aligned} \tag{3}$$

- Đây là phần giải thích đạo hàm của trọng số và bias đằng sau:

$$\frac{\partial J(w,b)}{\partial w} = \frac{1}{m}\sum_{i=0}^{m-1}(f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)} \tag{4}$$

$$\frac{\partial J(w,b)}{\partial b} = \frac{1}{m}\sum_{i=0}^{m-1}(f_{w,b}(x^{(i)}) - y^{(i)}) \tag{5}$$

Ta có hàm để tính Gradient:

```
def compute_gradient(x, y, w, b):
    """
    Computes the gradient for linear regression
    Args:
      x (ndarray (m,)): Data, m examples
      y (ndarray (m,)): target values
      w,b (scalar)    : model parameters
    Returns
      dj_dw (scalar): The gradient of the cost w.r.t. the par
ameters w
      dj_db (scalar): The gradient of the cost w.r.t. the par
ameter b
     """

    # Number of training examples
    m = x.shape[0]
    dj_dw = 0
    dj_db = 0

    for i in range(m):
        f_wb = w * x[i] + b
        dj_dw_i = (f_wb - y[i]) * x[i]
        dj_db_i = f_wb - y[i]
        dj_db += dj_db_i
        dj_dw += dj_dw_i
    dj_dw = dj_dw / m
    dj_db = dj_db / m
```

```
    return dj_dw, dj_db
```

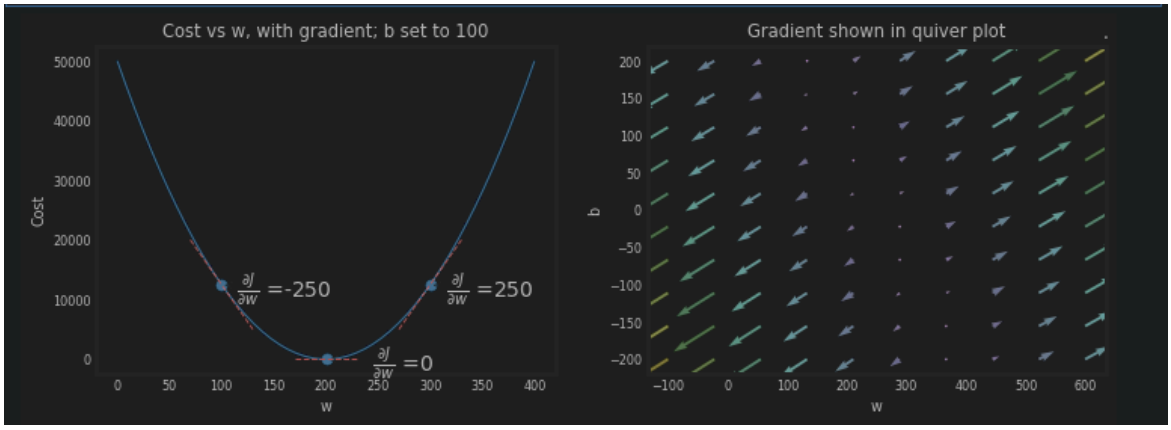
Note:

- m là số training example
- dj_dw, dj_db là đạo hàm tương ứng của w và b
- dj_dw_i và dj_db_i là để cộng dồn vào 2 biến đạo hàm trên, đây là của từng training example
- Sau đó mình chia cho m theo công thức và trả về 2 biến đạo hàm

---

Phần code này sử dụng hàmg được viết sẽ để minh hoạ độ dốc của Gradient

```
plt_gradients(x_train,y_train, compute_cost, compute_gradien
t)
plt.show()
```



Hàm Gradient Descent:

```
def gradient_descent(x, y, w_in, b_in, alpha, num_iters, cost
_function, gradient_function):
    """
    Performs gradient descent to fit w,b. Updates w,b by taki
ng
```

```
    num_iters gradient steps with learning rate alpha

    Args:
      x (ndarray (m,))  : Data, m examples
      y (ndarray (m,))  : target values
      w_in,b_in (scalar): initial values of model parameters
      alpha (float):     Learning rate
      num_iters (int):   number of iterations to run gradient
descent
      cost_function:     function to call to produce cost
      gradient_function: function to call to produce gradient

    Returns:
      w (scalar): Updated value of parameter after running gr
adient descent
      b (scalar): Updated value of parameter after running gr
adient descent
      J_history (List): History of cost values
      p_history (list): History of parameters [w,b]
      """

    w = copy.deepcopy(w_in) # avoid modifying global w_in
    # An array to store cost J and w's at each iteration prim
arily for graphing later
    J_history = []
    p_history = []
    b = b_in
    w = w_in

    for i in range(num_iters):
        # Calculate the gradient and update the parameters us
ing gradient_function
        dj_dw, dj_db = gradient_function(x, y, w , b)

        # Update Parameters using equation (3) above
        b = b - alpha * dj_db
```

```
        w = w - alpha * dj_dw

        # Save cost J at each iteration
        if i<100000:      # prevent resource exhaustion
            J_history.append( cost_function(x, y, w , b))
            p_history.append([w,b])
        # Print cost every at intervals 10 times or as many i
 terations if < 10
        if i% math.ceil(num_iters/10) == 0:
            print(f"Iteration {i:4}: Cost {J_history[-1]:0.2
 e} ",
                  f"dj_dw: {dj_dw: 0.3e}, dj_db: {dj_db: 0.3
 e}  ",
                  f"w: {w: 0.3e}, b:{b: 0.5e}")

    return w, b, J_history, p_history #return w and J,w histo
 ry for graphing
```

Note:

Một số tham số đầu vào:

1. x, y là dữ liệu huấn luyện (2 numpy array)
2. w_in và b_in là 2 giá trj khởi tạo của w và b (điểm xuất phát)
3. alpha là learning rate, chỉ thị bước đi của từng vòng lặp lớn hay nhỏ.
4. num_iters số vòng lặp cập nhât
5. cost_function hàm cost
6. gradient_function: hàm tính gradient

- Đầu tiên mình sẽ tính 2 biến đạo hàm bằng function đã được viết ở trên:

gradient_function(x, y, w , b)

- Sau đó tính gradient w và b hiện tại, cập nhất các tham số ấy
- Các phần .append của J_history và w_history là để tí nữa vẽ hình

- Câu điều kiện if là để in ra màn hình trong lúc chạy chương trình khoảng 10 lần (mỗi 10% tổng số vòng lặp)
- Sau khi tính xong mình sẽ trả kết quả: w, b đã được tối ưu, lịch sử của cost và tham số